



LOLBins? What about LOL AI?

Post-exploitation with Windows AI/ML

whoami

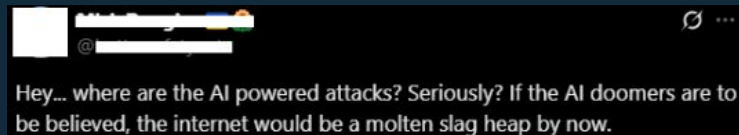
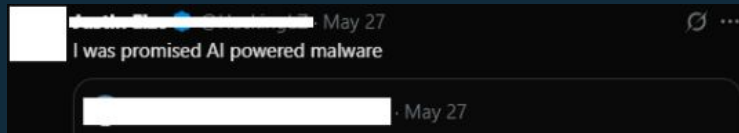
- Recovering crayon eater
- Post-exploitation nerd
- Reluctant tea drinker
- Casual OSRS player
- Marvel movie enjoyer



Senior Attack Engineer at Horizon3
@0xtriboulet

Why?

- AI is everywhere and on many more devices than you might think
- Microsoft continues to add additional AI/ML to the endpoint by default
- These features become post-exploitation capabilities waiting to be leveraged in post-exploitation



Agenda

- Background
- Research Methodology
- Post-ex POCs
 - ◆ BYO ML - piirate
 - ◆ OCR - ocular-rip
 - ◆ Speech recognition - parrot-rip
 - ◆ Copilot - silent_copilot
- Conclusion

Background

AI/ML on Windows

Post-Exploitation



Requirements:

- Must support reflective loading
- Inference must be fast (on the order of seconds)
- Binary format must be small-ish (~10MBs max)
- Maximize organic OS capabilities

AI in Post-Ex

- Ingress arbitrary models for on-target inference (PII identification)
- Identify sensitive information in images and image-based documents (OCR)
- Listen on a microphone for keywords (Speech recognition)
- Agentic uplift with LLMs (on-the-fly script generation). **Mostly** out of scope

AI/ML in Windows

- Windows includes (some) ML API since Windows 10 version 1809
 - Built-in OCR support predates the API
 - Speech recognition
 - Copilot
-
- NPUs support additional features and APIs. Out of scope

ML API

- Windows 10 (2018) and “built into the latest versions of Windows”
- CPUs, GPUs, and AI accelerators
- Uses ONNX standard <https://onnx.ai/>
- Moderately well documented <https://github.com/MicrosoftDocs/windows-ai-docs>
- **Microsoft.Windows.AI.MachineLearning**

Windows.AI.MachineLearning Namespace

📌 Important

For the latest documentation about Windows Machine Learning, see [What is Windows ML](#). That documentation describes APIs that are in the `Microsoft.Windows.AI.MachineLearning` namespace, which ships in the Windows App SDK. Those APIs supersede the ones documented here, which are in the `Windows.AI.MachineLearning` namespace, and were shipped in 2018.

Enables apps to load machine learning models, bind features, and evaluate the results.

OCR

- Optical Character Recognition
 - ◆ Turns images into text
- OCR API has shipped with Windows since Windows 10 (2015)
 - ◆ Entirely local inference
 - ◆ Language support enabled through Windows settings
- **Windows.Media.Ocr** Namespace

Remarks

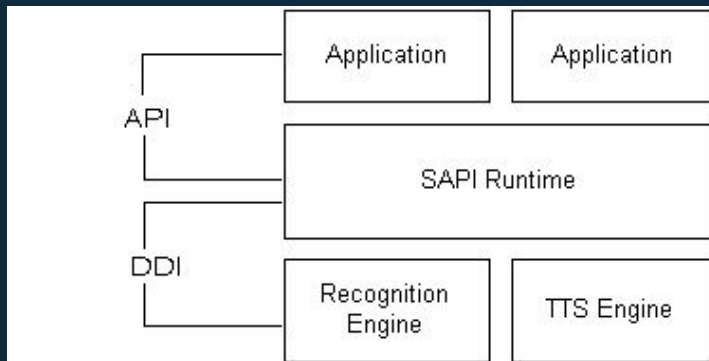
APIs in the Windows.Media.Ocr namespace are only supported for desktop apps with *package identity*. This means that the app is installed and run from an MSIX package. For more information, see [Features that require package identity](#). For information about calling these APIs from a desktop app, see [Call Windows Runtime APIs in desktop apps](#).

<https://learn.microsoft.com/en-us/uwp/api/windows.media.ocr?view=winrt-28000>

This is **not** a fact

Speech recognition

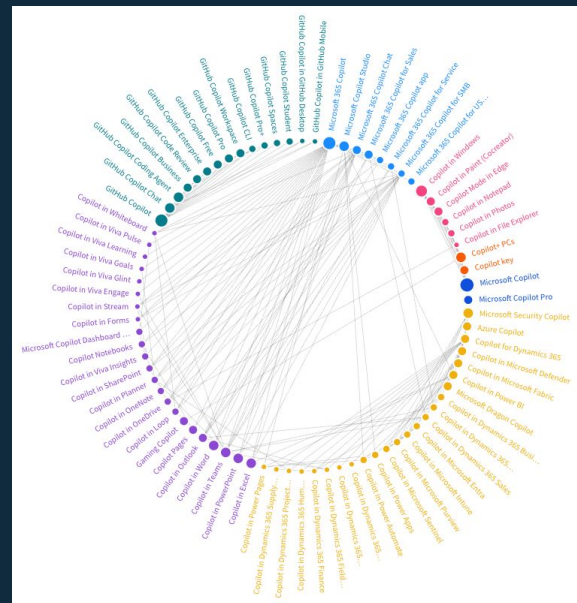
- Speech recognition was unified into a coherent API in Windows Vista
 - ◆ Prior to this, different Windows components (Office) already supported speech recognition
- Microsoft Speech API (SAPI) to provide a high-level interface
- **ISpRecoContext** is the main COM interface



[https://learn.microsoft.com/en-us/previous-versions/windows/desktop/ms720151\(v=vs.85\)](https://learn.microsoft.com/en-us/previous-versions/windows/desktop/ms720151(v=vs.85))

Copilot

- What even is it?
 - ◆ ~75 distinct products/services share "Copilot"
- Software interface for Microsoft's LLM backend
- **M365Copilot.exe**
 - ◆ Receives user input
 - ◆ Performs API requests
 - ◆ Receives response and present it to user



Research Methodology

Past and Present

AI in Post-ex (2025)

- BYO AI/ML Model as post-ex binary in a C2
- Reference C2 documentation, Windows documentation, Microsoft code examples
- ChatGPT could help port some of the code from C# to C++
- A ton of manual refining and testing to achieve expected functionality



The image shows the top section of the Black Hat USA 2025 website. It features a dark background with vibrant, abstract purple and pink light trails. On the left, the 'black hat' logo is displayed in white, with 'USA 2025' underneath. To the right, a white button says 'REGISTER NOW'. Below this, the dates 'AUGUST 2-7, 2025' and the location 'MÁNDALAY BAY / LAS VEGAS' are listed. A horizontal navigation bar contains links for ATTEND, TRAININGS, BRIEFINGS, ARSENAL, SUMMITS, FEATURES, BUSINESS HALL, and SPONSORS. Below the navigation bar, a note states 'All times are Pacific Time (GMT/UTC -7h)'. On the left side of the main content area, there are two blue buttons: 'ALL SESSIONS' and 'PRESENTERS'. The main content area features a featured session titled 'Next-Generation Post-Exploitation in Cobalt Strike' by Steve Salinas. The session details are: Date: Wednesday, August 6 | 10:00am-10:55am (Business Hall, Arsenal Station 5) and Track: Malware.

black hat
USA 2025

REGISTER NOW

AUGUST 2-7, 2025
MÁNDALAY BAY / LAS VEGAS

ATTEND TRAININGS BRIEFINGS ARSENAL SUMMITS FEATURES BUSINESS HALL SPONSORS

All times are Pacific Time (GMT/UTC -7h)

ALL SESSIONS

PRESENTERS

Next-Generation Post-Exploitation in Cobalt Strike

Steve Salinas

Date: Wednesday, August 6 | 10:00am-10:55am (Business Hall, Arsenal Station 5)

Track: Malware

AI in Post-ex (2026)



AI in Post-ex (2026)

→ Steps:

1. Find an inference provider where models allow for post-ex research (Ollama)
2. Plug that inference provider into an agent harness (Pi)
3. Give the model of choice (GLM 5.2) ssh access to a Windows virtual machine
4. ??????
5. PROFIT?!?!?



AI in Post-ex (2026)

→ Sort of profit...



AI in Post-ex (2026)

→ Sort of profit...

- ◆ Model needs strict standards, documentation, and manual validation
 - -std=C89 -Wall -Wextra -Werror
 - Agentic RE + .winmd files were very powerful in driving development
- ◆ Post-exploitation requirements != software development requirements
- ◆ Refining POCs and validating the findings still a requirement, still slow-ish
- ◆ **Models are eager to write code to solve problems. Sometimes the actual answer is less code**



Post-exploitation POCs

Activating instant-kill

Goals

- ◆ Standalone C89 binaries with minimal dependencies
 - C API for Windows ML
- ◆ BYO ML requires ONNX
 - .onnx model's opset version must be supported by the onnxruntime.dll on Windows 11 25H2
 - Onnxruntime C API
- ◆ Inference primitives must be general available on Windows systems
- ◆ Drive development with GLM-5.2/3 workflow
 - Lead to some interesting outcomes

BYO ML: Piirate



wmain

ort_init

scan_dir

scan_path

pii_tok_encode_word

infer_windows

piirate - uses Gliner-INT8 model to recursively scan documents for PII when given a root directory

Use arbitrary classification model from HuggingFace

Requires that we implement entire tokenization and encoding that the model expects

- Models do not consume raw text, they consume encoded tokens

BYO ML: Piirate



wmain

piirate - uses Gliner-INT8 model to recursively scan documents for PII when given a root directory

ort_init

ort_init - dynamically initializes inference engine with onnxruntime.dll, stores relevant API in a global g_api struct

scan_dir

scan_path

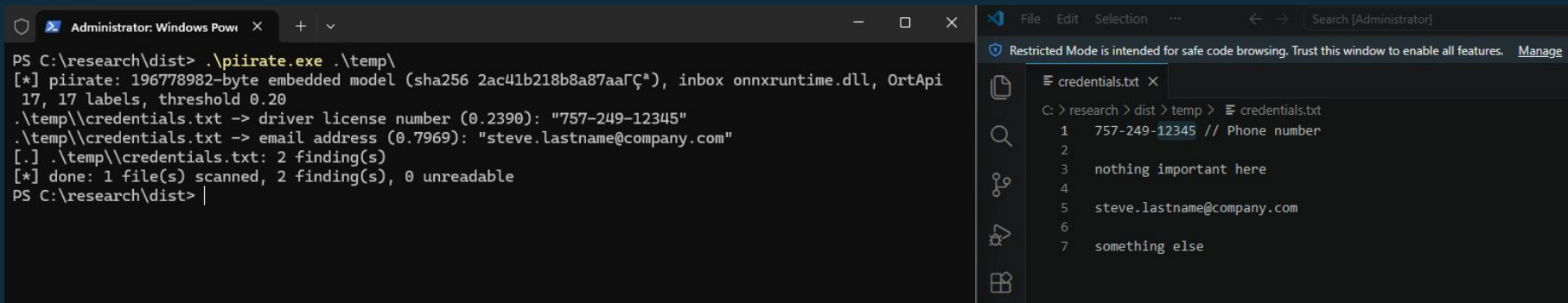
pii_tok_encode_word - takes the words extracted from documents and performs tokenization + encoding

pii_tok_encode_word

infer_windows - converts encoded buffer into tensor buffer of the expected shape, and uses g_api→run() to run model inference over the tensor

infer_windows

BYO ML: Piirate



The screenshot displays two windows. The left window is a PowerShell terminal titled 'Administrator: Windows PowerShell' showing the execution of the 'piirate.exe' tool. The right window is a code editor titled 'Restricted Mode is intended for safe code browsing. Trust this window to enable all features. Manage' showing the contents of a file named 'credentials.txt'.

```
PS C:\research\dist> .\piirate.exe .\temp\  
[*] piirate: 196778982-byte embedded model (sha256 2ac41b218b8a87aa7c^), inbox onnxruntime.dll, OrtApi  
17, 17 labels, threshold 0.20  
.\temp\credentials.txt -> driver license number (0.2390): "757-249-12345"  
.\temp\credentials.txt -> email address (0.7969): "steve.lastname@company.com"  
[.] .\temp\credentials.txt: 2 finding(s)  
[*] done: 1 file(s) scanned, 2 finding(s), 0 unreadable  
PS C:\research\dist> |
```

The code editor shows the following content in 'credentials.txt':

```
C: > research > dist > temp > credentials.txt  
1 757-249-12345 // Phone number  
2  
3 nothing important here  
4  
5 steve.lastname@company.com  
6  
7 something else
```

Information correctly found, incorrectly classified phone number as
driver license number

OCR: ocular-rip



wmain

run_ocr

get_factory(File/Decoder/Engine)

RecognizeAsync(engine,bmp) → resObj

result→get_Text()

ocular-rip - takes a path to a single image, extracts text in that image, prints to stdout

Rather than bringing our own model, use Windows' OCR model

We do *not* need to implement any encoding mechanism

We *do* need to convert the image into a bitmap

OCR: ocular-rip



wmain

run_ocr

get_factory(File/Decoder/Engine)

RecognizeAsync(engine,bmp) → resObj

result→get_Text()

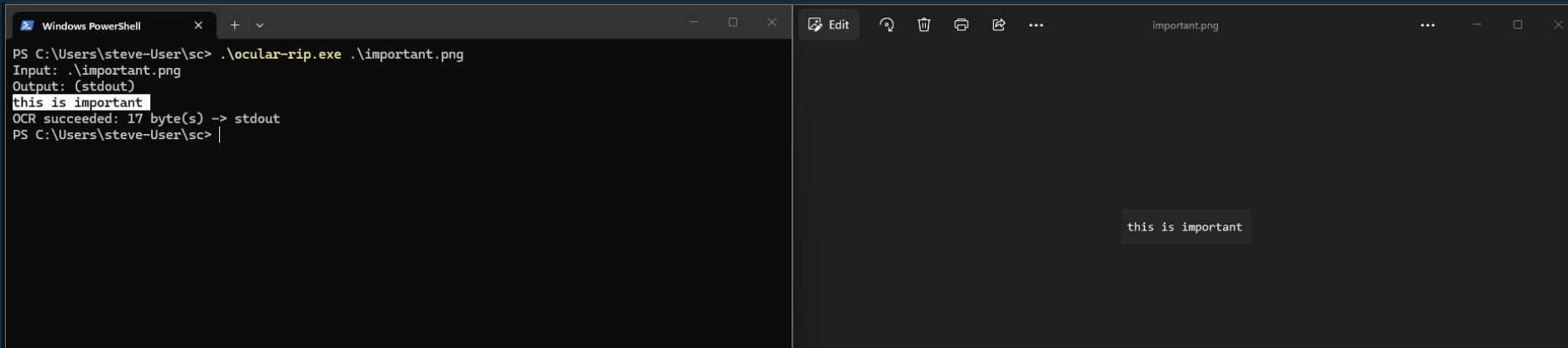
ocular-rip - takes a path to a single image, extracts text in that image, prints to stdout

get_factory - wrapper for call to RoGetActivationFactory for FileRandomAccessStream, BitmapDecoder, OcrEngine

RecognizeAsync - method of engine object that recognizes text in bitmaps

result→get_Text() - uses the result object output by the engine

OCR: ocular-rip



Text extracted from the image on the right

Speech: parrot-rip



wmain

parrot-rip - takes one sample via the default microphone, transcribes it with SAPI5, prints text to stdout

cap_wavein

Similarly, we do not need to bring our own model, implement encoding, or any AI/ML primitives

sapi_process_stream

sapi_take_event

Windows provides all the interfaces we need

Speech: parrot-rip



wmain

parrot-rip - takes one sample via the default microphone, transcribes it with SAPI5, prints text to stdout

cap_wavein

cap_wavein - wrapper for MME waveIn (open/start on default device)

sapi_process_stream

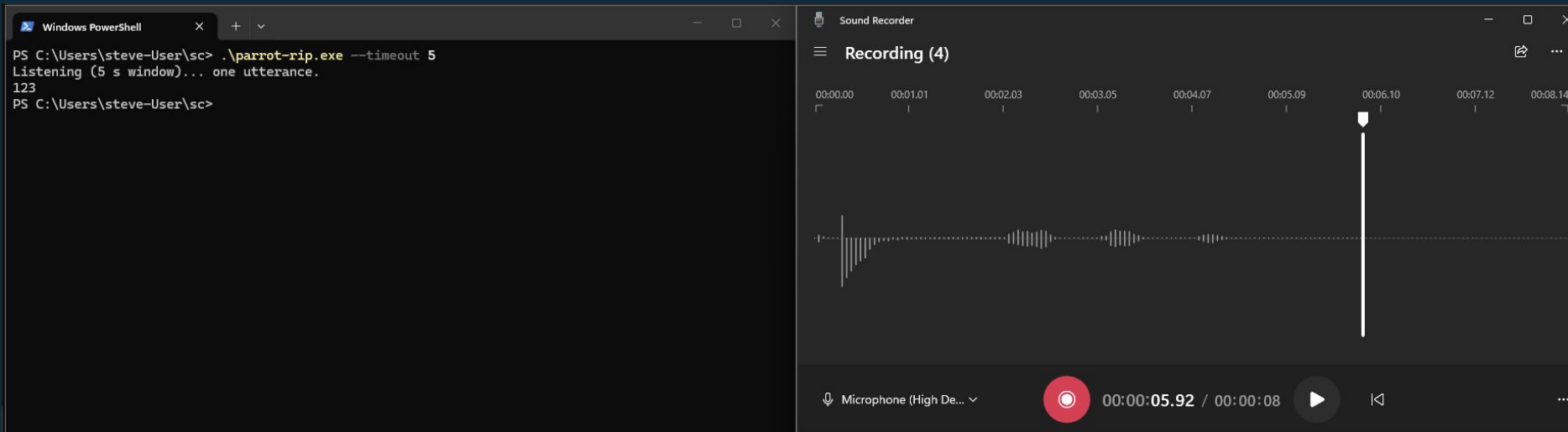
sapi_process_stream - wrapper for CoCreateInstance of SpInprocRecognizer and waits on the SAPI notify event

sapi_take_event

sapi_take_event - method on the event loop that handles SPEI_RECOGNITION events and pulls out the text

r→lpVtbl→GetText - uses the ISpRecoResult object output by the engine to get the recognized string

Speech: parrot-rip



Text is extracted from microphone. Sound recorder is a visual aid,
not a requirement.

Copilot: silent_copilot



wmain

scan_all_processes

sapi_process_stream

ensure_session_material

try_token

ws_send/recv_msg

silent_copilot - sends a prompt to M365 Copilot (substrate.office.com), prints the reply to stdout. **Requires an existing chat session with M365 Copilot**

JWE access tokens - begin `eyJhbGciOiJkaXli` = `{ "alg": "dir" }`
session-tenant GUID - hex-tail `aaaaaaaaaaaaa`

Client-generated GUIDs - sess, crid, xsid, rid, trace that can be freshly random per attempt

Copilot is a legitimate Windows cloud service, silent_copilot is a client on a legitimate Windows client ⇒ raw TCP + Windows Schannel

Copilot: silent_copilot



wmain

scan_all_processes

sapi_process_stream

ensure_session_material

try_token

ws_send/rcv_msg

silent_copilot - sends a prompt to M365 Copilot (substrate.office.com), prints the reply to stdout

scan_all_processes - wrapper for scanning M365Copilot/msedgewebview2 memory via ReadProcessMemory

ensure_session_material - harvests JWE access tokens + session/tenant GUID from process memory, deduped

try_token - one attempt: opens a Schannel (SSPI) TLS connection and drives the WebSocket chat stream, parses the reply

ws_send/rcv_msg - WebSocket frame send/recover the TLS connection

Copilot: silent_copilot

GET /m365Copilot/Chathub/**/84df9e7f-e9f6-40af-b435-aaaaaaaaaaaaa?****access_token=eyJhbGciOiJkaXI...**

session-tenant GUID

JWE access tokens

Copilot: silent_copilot

```
Windows PowerShell
PS C:\Users\steve-User\sc> .\silent_copilot.exe --prompt 'write a powershell snippet that enumerates running processes'
[*] silent_copilot: prompt captured
PS C:\Users\steve-User\sc> [*] STEN=84df9e7f-e9f6-40af-b435-aaaaaaaaaaaa tokens=20 vars=2725

[*] trying token, tail=Mew7L509qSVQn8f31SmjgiAN
upgrade rejected
[*] trying token, tail=dVLDqzRNiaXKwHQ9XgOCrN4E
```powershell
Enumerate all running processes
Get-Process |
 Sort-Object ProcessName |
 Select-Object Id, ProcessName, CPU, WorkingSet

Alternatively, display a formatted table
Get-Process |
 Sort-Object ProcessName |
 Format-Table Id, ProcessName, CPU, WorkingSet -AutoSize
```
To enumerate and include the executable path when available:
```powershell
Get-CimInstance Win32_Process |
 Select-Object ProcessId, Name, ExecutablePath |
 Sort-Object Name
```
[+] reply captured
```

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\steve-User> Get-CimInstance Win32_Process |
>>     Select-Object ProcessId, Name, ExecutablePath |
>>     Sort-Object Name

ProcessId Name                               ExecutablePath
-----
3496  AggregatorHost.exe                  C:\WINDOWS\system32\ApplicationFrameHost.exe
12076 ApplicationFrameHost.exe             C:\WINDOWS\system32\backgroundTaskHost.exe
7980  backgroundTaskHost.exe              C:\WINDOWS\system32\backgroundTaskHost.exe
11204 cmd.exe
11020 conhost.exe
976   conhost.exe                         C:\WINDOWS\system32\conhost.exe
5924  CrossDeviceResume.exe               C:\WINDOWS\SystemApps\MicrosoftWindows.Client.CBS_cw5n1h2txyewy\CrossDeviceResu
me.exe
648   csrss.exe
728   csrss.exe
7864  ctfmon.exe
3372  dasHost.exe
```

Copilot session silently hijacked and used to generate powershell scripts on the fly

Limitations

- BYOML requires that you implement and bring everything yourself
- ocular-rip and parrot-rip rely on older legacy APIs
- silent_copilot requires an existing active Copilot chat
- AI code is moderately good; only functionality reviewed and validated

Future Work

- Operationalizing the POCs into post-exploitation formats
 - ◆ BOF-PE <https://github.com/NetSPI/BOF-PE>
- Continuing to reverse M365Copilot.exe and its protocols
- Study what MS is adding to the endpoint
 - ◆ More AI == More Post-Ex Capability

Conclusion

- AI/ML on the endpoint is here and taking many shapes
 - ◆ BYOML
 - ◆ OCR
 - ◆ Speech Recognition
 - ◆ LLM Session Hijacking

Questions?